

Terminal-First Multimedia on openSUSE

How far the terminal goes, and where it stops being one

Yuki Otsuka

Japan openSUSE User Group

Yogyakarta | October 2026

Profile

Yuki Otsuka (*akusto*)

- Member of the Japan openSUSE User Group
- Graduate student at the Japan Advanced Institute of Science and Technology (JAIST)
- Daily openSUSE user — mainly Tumbleweed, also Leap 16

GitHub: [4kusto](#) | Pijul Nest: [4kusto](#) | yuki.akusto@gmail.com

Today's route

- 1 A terminal as a workspace**
- 2 bcon on the Linux console**
- 3 Images, video, and the Web**
- 4 What's missing, and what's next**

A terminal as a workspace

Why start in the terminal?

Most of my working day already happens in a terminal:

- I edit, build, and test there
- I use VCS and read logs there
- I manage files and SSH sessions there

My question: can this become the main openSUSE workspace?

The openSUSE experiment

- Log in on an openSUSE Linux TTY
- Start bcon, without X11 or Wayland
- Try panes, Japanese text, images, and terminal apps
- Compare with my usual graphical desktop

This changes the display stack, not the basic workflow.

Case study: my bcon setup on openSUSE Tumbleweed.

bcon on the Linux console

Why bcon?

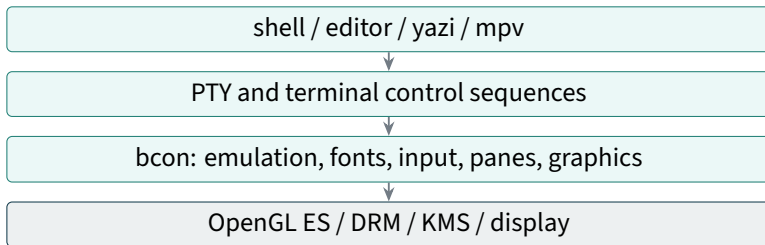
I wanted all of this in one terminal, on the bare console:

- runs directly on the Linux console
- GPU rendering: OpenGL ES over DRM/KMS
- panes and tabs
- both Sixel and Kitty graphics
- fcitx5 integration for Japanese input

That combination is why I chose bcon.

Upstream: <https://github.com/sanohiro/bcon>

Where the pixels come from



No X11 server or Wayland compositor is needed for this path.

Schematic of the roles involved; not a trace of bcon internals.

Display environment

openSUSE Tumbleweed • bcon 1.4.0

- Linux 7.2.6-1-default
- no DISPLAY, no Wayland
- /dev/dri/card1 (nvidia)
- 382 cols x 105 rows

Terminal and input environment

openSUSE Tumbleweed • bcon 1.4.0

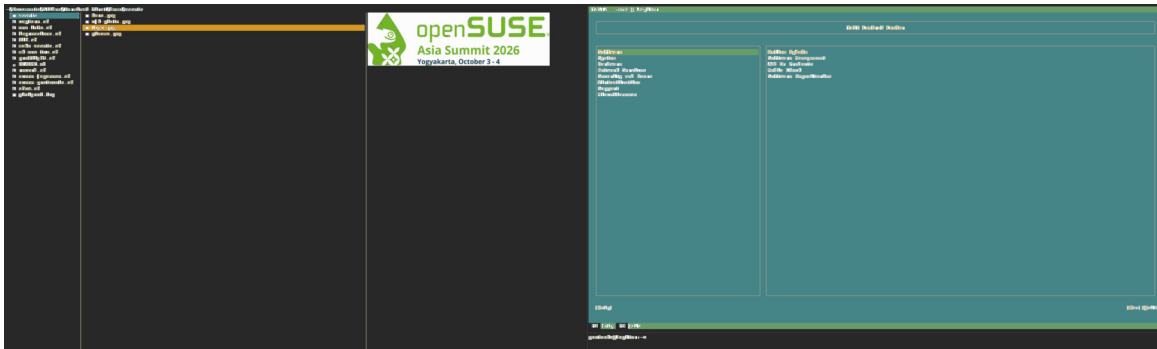
- TERM=xterm-256color
- 10 × 20 px per cell
- CJK Noto Sans CJK JP
- fcitx5 5.1.17 running

Apps can ask the terminal what it supports

query	reply
primary device attributes	<code>^[[?62;1;4;22;29c</code>
text area, pixels	<code>^[[4;2100;3820t</code>
text area, cells	<code>^[[8;105;382t</code>
Kitty graphics	<code>^[_Gi=31;OK^[\\</code>

bcon replies with its size and supported terminal features.

Panes make a usable work area



yazi previewing through Kitty graphics, YaST in ncurses, a shell — one bcon window, no compositor.

Japanese needs display and input

Read

- TrueType fonts and CJK glyphs
- Font fallback and cell width

Write

- fcitx5 through D-Bus
- IME setup and a user session

Using Japanese requires both correct display and IME input.

bcon features and setup: <https://github.com/sanohiro/bcon>

Images, video, and the Web



An image inside a file manager



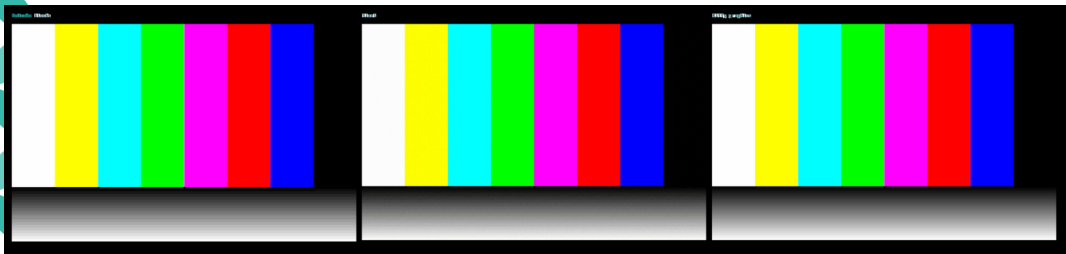
yazi previewing bars . png — the test card the next slide sends three ways.

Same image, three transports

Unicode blocks

Sixel

Kitty graphics



At this size, they look almost the same.

The difference is how the image gets there

Unicode blocks

- coloured text cells
- approximation, not raster

Sixel

- raster, as escape sequences
- DEC terminals, 1980s

Kitty graphics

- much newer image protocol
- used by modern terminal apps

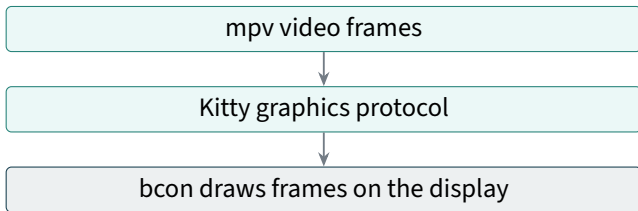
The visible result can be similar even when the transport is very different.

Why support both Sixel and Kitty?

- Sixel has been around for decades — existing terminal software still uses it
- Kitty graphics is much newer — a growing set of modern terminal applications use it
- bcon supports applications using either protocol, in the same window

I do not have to choose applications based on one graphics protocol.

From preview to video



`mpv -vo=kitty video.mp4`

Controlling a player is different from displaying its frames here.

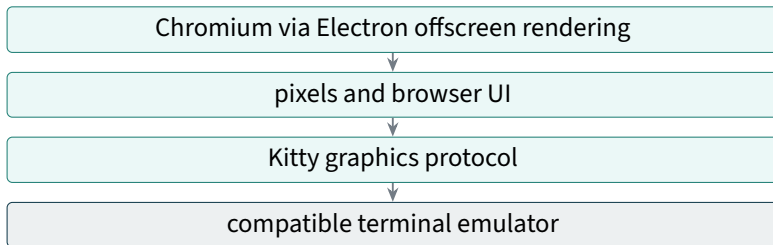
Supported workflow documented by bcon upstream. No playback benchmark claimed.

Then there is the modern Web

- w3m keeps a page as terminal text
- Articles and documentation often work well
- Modern pages need layout, script, media, and a rendering engine

Text browsers are useful, but they are not the whole Web.

terminal-browser takes another route



The Web stays graphical; the terminal carries its pixels.

Upstream design: <https://github.com/zenbu-labs/terminal-browser>

The page's pixels do arrive in bcon



Captured with bcon's own screenshot key.

What that capture shows — and what it does not

- Shows: rendered page pixels reach bcon
- Does not show: input events reaching Chromium
- Does not show: Electron startup needs on a bare TTY

This capture proves display, not interaction.

What's missing, and what's next



Where this setup still falls short

- Graphics and input support vary between terminal emulators
- Clipboard, notifications, and accessibility are not as integrated as on a desktop
- bcon currently supports one active monitor and no multi-seat setup

This works, but it is not a full desktop replacement yet.

bcon upstream lists one active monitor and no multi-seat support.

Terminal-first vs. a graphical desktop

Terminal-first / bcon

- keyboard-oriented workflow
- terminal apps compose well
- one main visual workspace
- graphics via Sixel and Kitty
- no X11/Wayland in this path

X11 / Wayland desktop

- GUI apps work naturally
- clipboard and notifications
- accessibility integration
- mouse-driven workflows
- multi-window, multi-monitor
- broader app compatibility

This is about workflow, not performance

- I have not benchmarked this setup against X11 or Wayland
- My reason for using it is the workflow, not benchmark numbers

So — what does this workflow actually give me?

Why I actually like working this way

- One keyboard-oriented visual workspace
- Fewer window and application switches
- Multimedia appears when I ask for it, in the same workspace

For me, fewer visual interruptions make it easier to stay focused.

The question from “Why start in the terminal?” — for my own daily work, the answer is: mostly, yes.

This is an integration story

- bcon — console terminal and graphics
- yazi — file management and image previews
- mpv — video
- fcitx5 — Japanese input
- terminal-browser — rendered Web content
- openSUSE / OBS — packaging and reproducibility

These are upstream OSS projects. My work here is selecting, testing, combining, and packaging them on openSUSE.

Packaging is part of the experiment

- Next: prepare a bcon RPM and check dependencies
- Evaluate terminal-browser's Linux build and redistribution
- Document a reproducible openSUSE setup

OBS could make the setup repeatable.

Conclusion

- With Sixel and Kitty graphics, a terminal can show much more than text.
- bcon brings both directly to the Linux console.
- This does not replace a graphical desktop — but for my workflow, it goes surprisingly far.

All of this is existing open source, combined on openSUSE.

Thank you!

Yuki Otsuka

Japan openSUSE User Group

yuki.akusto@gmail.com

After the session, feel free to come by
— I am happy to show the demo any
time.



openSUSE®

Asia Summit 2026

Yogyakarta, October 3 - 4